4NF Explanation

The BCNF design is already in 4NF: every independent list sits in its own table, so no further decomposition was needed.

Multi-valued dependencies identified

We looked for tables where one entity has two or more independent lists of facts. In the raw data these were:

- A worker's skills and certifications on a project: `(ProjectID, WorkerName) ↠ WorkerSkill` and `(ProjectID, WorkerName) ↠ Certification`.
- A supplier's equipment and materials on a project.
- A supplier's phone numbers and city.

Independent lists that were mixed together

Skills with certifications, equipment with materials, and supplier phones with supplier details. If any pair sat in one table, every item in one list would have to be paired with every item in the other, which creates redundant rows.

How we separated them

Each list has its own table: `Assignment_Skills`, `Assignment_Certifications`, `Project_Supplier_Equipment`, `Project_Supplier_Materials` and `Supplier_Phones`. These separations were done in earlier stages, so at 4NF we checked them instead of splitting again. We ran `check_4nf.py`, which tries every way of splitting each table into two projections and checks whether joining them reproduces exactly the original rows.

Why skills and certifications stay at the (project, worker) level

The data shows that a worker's skills and certifications change from project to project. For example, Harvey Specter's skills differ across P001, P005 and P007, and Rachel Zane's certifications differ between projects. A worker-level table such as `Worker_Skills(WorkerName, Skill)` would lose which skills applied on which project, so we kept the project context.

Primary keys of the list tables

Table	Primary key
Assignment_Skills	(ProjectID, WorkerName, WorkerSkill)
Assignment_Certifications	(ProjectID, WorkerName, Certification)
Project_Supplier_Equipment	(ProjectID, SupplierName, EquipmentUsed)
Project_Supplier_Materials	(ProjectID, SupplierName, MaterialSupplied)
Supplier_Phones	SupplierPhone

Foreign keys: `(ProjectID, WorkerName)` references `Project_Assignments`, `(ProjectID, SupplierName)` references `Project_Suppliers`, and `SupplierName` references `Suppliers`.

Flagged results from the script

The script reported `MaterialUnitCost ↠ MaterialSupplied | ProjectID` in `Project_Materials`. We rejected it as a sample-data coincidence, because almost every price in the data belongs to one material (Steel Beams at 500 on P001 and P004 is the only repeated price). A price does not determine a material or a project, and the real key of the table is `(ProjectID, MaterialSupplied)`, so we did not split the table. The script skipped Projects because it has a single key with only single-valued attributes, which we confirmed by hand. The OK lines for Clients, Workers and Project_Equipment are trivial because the determinant is a key.

Why the final design is in 4NF

It is already in BCNF. Every independent list sits in its own table, and in every table each non-trivial multi-valued dependency has a candidate key as its determinant. No table mixes two independent lists, so no further decomposition was needed.